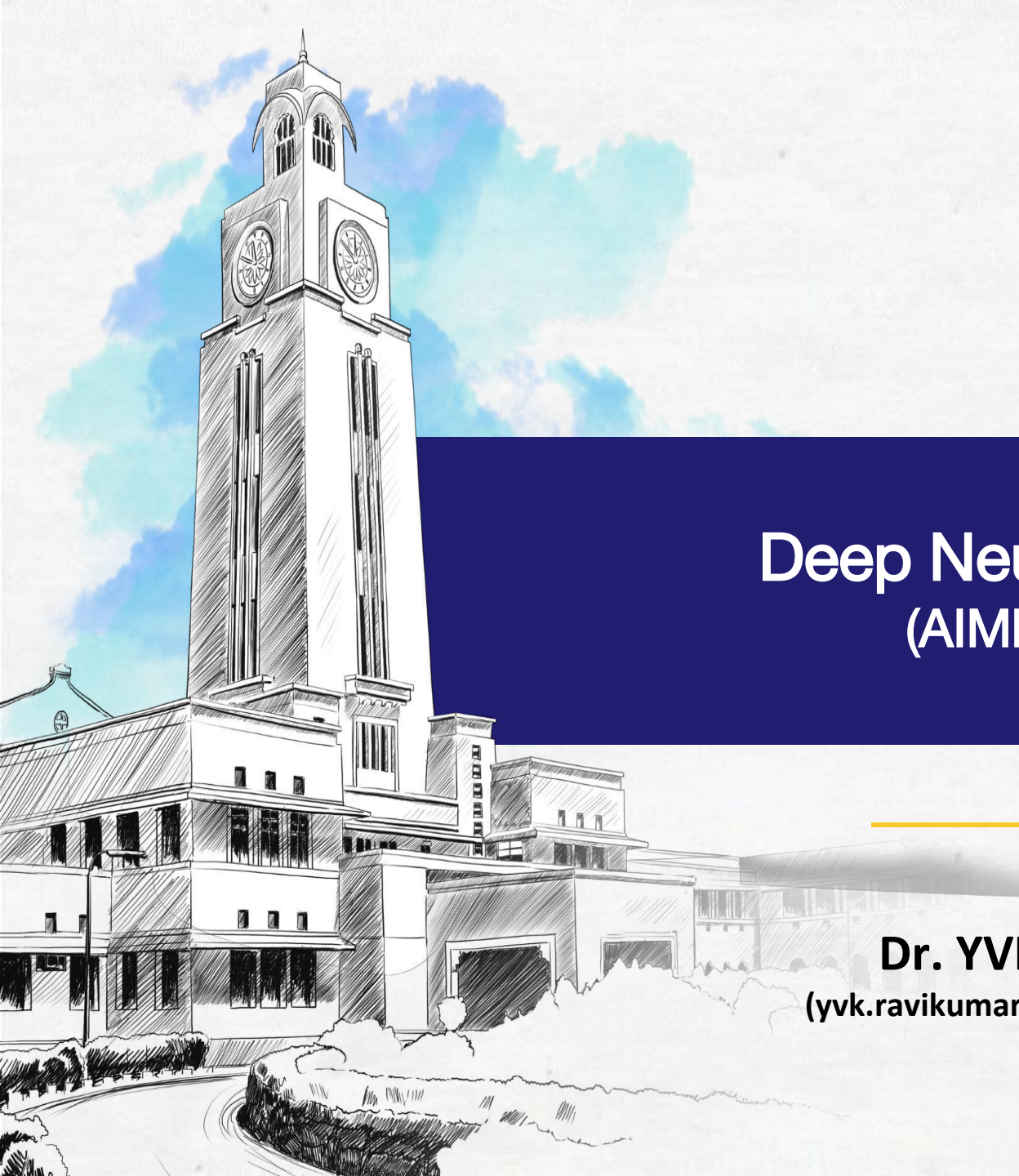




BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Deep Neural Networks (AIML CZG511)

Dr. YVK Ravi Kumar
(yvk.ravikumar@pilani.bits-pilani.ac.in)



Session 3
(10th May, 2026)



MODULE # 3 :
LINEAR NEURAL NETWORKS FOR REGRESSION

TABLE OF CONTENTS

- ✓ REGRESSION
 - ✓ LINEAR REGRESSION
 - ✓ THE LINEAR MODEL: SINGLE NEURON
 - ✓ WORKED EXAMPLE
 - ✓ EVALUATION
 - ✓ PYTHON IMPLEMENTATION IMPLEMENTATION TIPS
 - ✓ SUMMARY
- 
- The logo of Birla Institute of Technology & Science (BITS) is visible in the background. It is a circular emblem with a gear-like border. Inside the circle, there is a stylized 'V' shape with a flame-like element at the top. The text 'BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE' is written around the top inner edge, and 'PILANI' is at the bottom.

WHAT IS REGRESSION?

❑ DEFINITION

Regression is a supervised learning task that predicts a continuous-valued output based on input features.

○ Key Characteristics:

- Input: Feature vector $\mathbf{x} \in \mathbb{R}^d$
- Output: Real-valued target $y \in \mathbb{R}$
- Goal: Learn a function $f: \mathbb{R}^d \rightarrow \mathbb{R}$ such that $y \approx f(\mathbf{x})$

EXAMPLES OF REGRESSION PROBLEMS

➤ House Price Prediction

- Input: Size, location, number of rooms
- Output: Price in dollars

➤ Temperature Forecasting

- Input: Historical weather data, season, humidity
- Output: Temperature in degrees

➤ Stock Price Prediction

- Input: Historical prices, trading volume, market indicators
- Output: Future stock price

WHAT IS LINEAR REGRESSION?

DEFINITION

- Linear regression assumes the output is a linear combination of the input features.

➤ Mathematical Form:

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d$$

where:

$\mathbf{x} = [x_1, x_2, \dots, x_d]^T$ are the input features

$\mathbf{w} = [w_1, w_2, \dots, w_d]^T$ are the weights

w_0 is the bias term

EXAMPLES OF LINEAR REGRESSION

➤ Salary Prediction

$$\text{Salary} = w_0 + w_1 \times \text{Years of Experience}$$

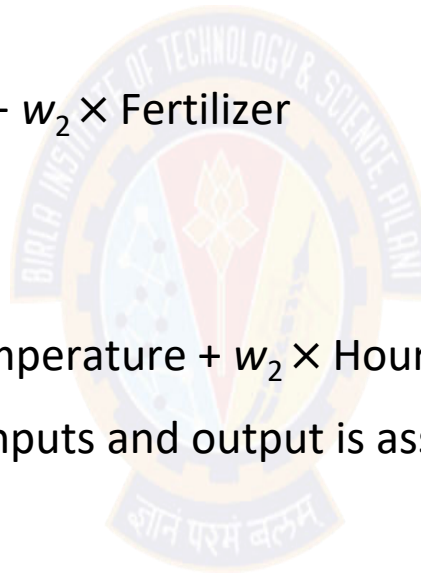
➤ Crop Yield Prediction

$$\text{Yield} = w_0 + w_1 \times \text{Rainfall} + w_2 \times \text{Fertilizer}$$

➤ Energy Consumption

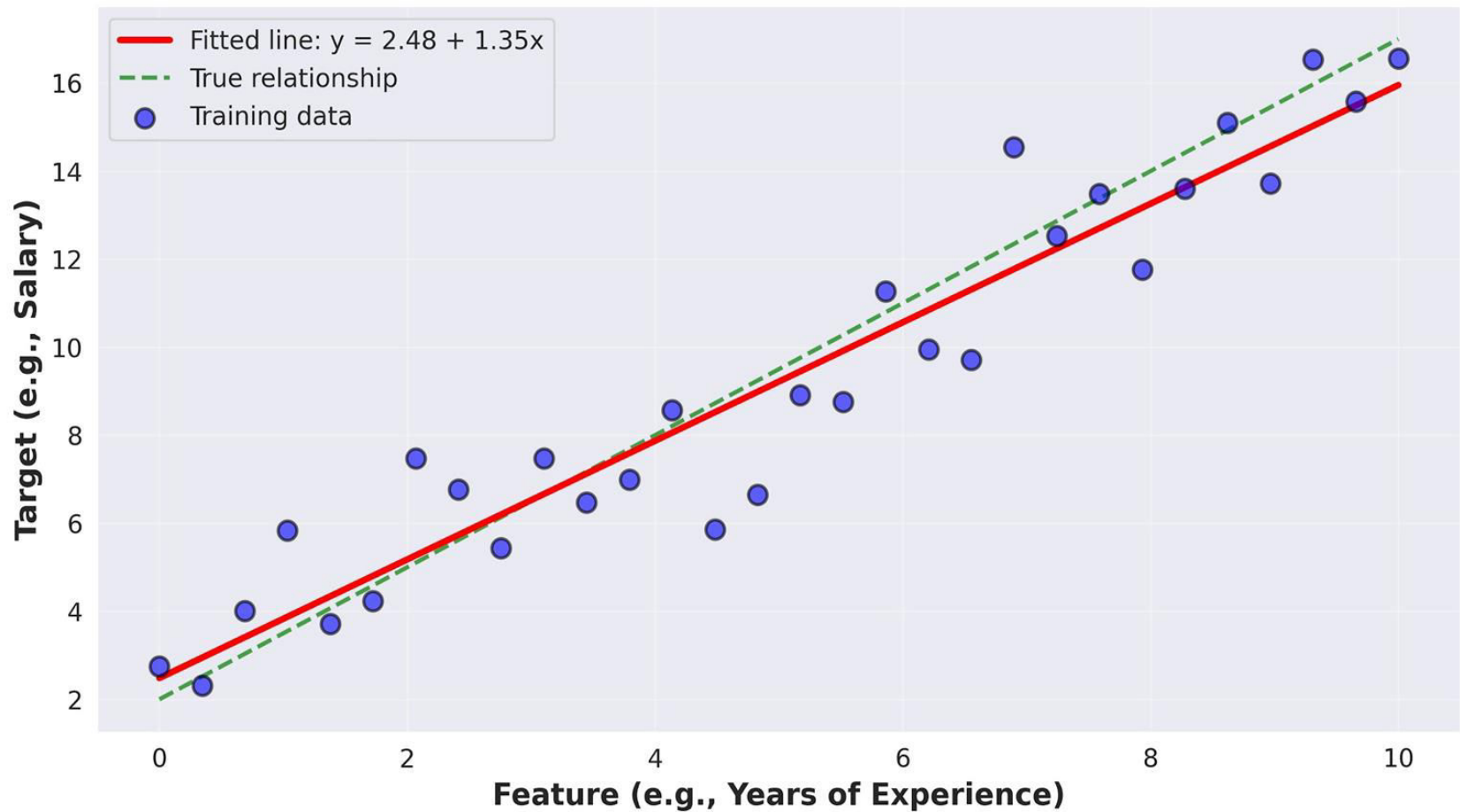
$$\text{Energy} = w_0 + w_1 \times \text{Temperature} + w_2 \times \text{Hour} + w_3 \times \text{Day}$$

The relationship between inputs and output is assumed to be **linear**.



VISUAL EXAMPLE: 2D LINEAR REGRESSION

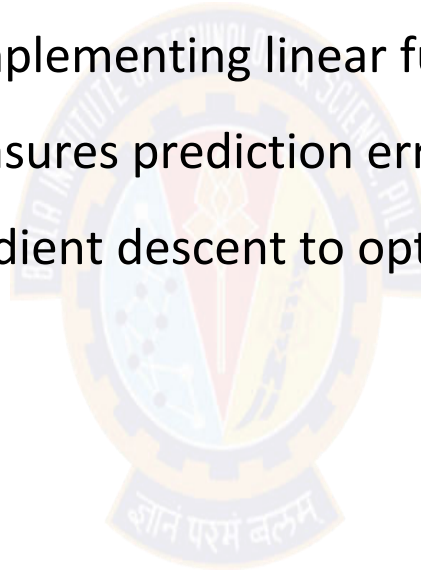
2D Linear Regression: Scatter Plot with Fitted Line



THE FOUR COMPONENTS

A complete machine learning system consists of:

- **Data:** d -dimensional input vectors and target outputs
- **Model:** Single neuron implementing linear function
- **Objective Function:** Measures prediction error
- **Learning Algorithm:** Gradient descent to optimize weights



MATRIX FORM OF DATA

For efficient computation, we organize data into matrices:

Input Data = Augmented Features:

Add bias term: $\tilde{\mathbf{x}}^{(i)} = [1, x_1^{(i)}, \dots, x_d^{(i)}]^T$

Each row = one training example

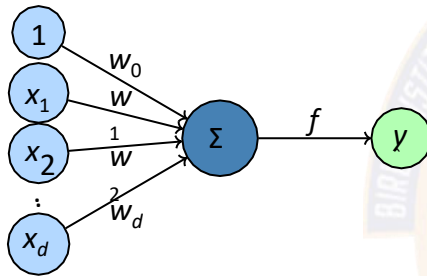
Design Matrix: $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$ $\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & \dots & x_d^{(N)} \end{bmatrix}$

Target Vector: $\mathbf{y} \in \mathbb{R}^N$ $\mathbf{y} = [y^{(1)} \ y^{(2)} \ \dots \ y^{(N)}]^T$

Weight Vector: $\mathbf{w} \in \mathbb{R}^{d+1}$ $\mathbf{w} = [w_0 \ w_1 \ w_2 \ \dots \ w_d]^T$

LINEAR MODEL AS A SINGLE NEURON

- Model linear regression as a single artificial neuron:



- Input: d -dimensional feature vector $\mathbf{x} \in \mathbb{R}^d$
- Output: Single predicted value $\hat{y} \in \mathbb{R}$

LINEAR MODEL PREDICTION

Model: A single neuron computes a weighted sum and applied identity activation function.

Prediction for input $\mathbf{x}$:

$$\hat{y} = f(\mathbf{w}^T \mathbf{x}) = f(w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d) \quad (2)$$

where:

- $\mathbf{w} = [w_0, w_1, \dots, w_d]^T \in \mathbb{R}^{d+1}$ are the model parameters
- $\mathbf{x} = [1, x_1, \dots, x_d]^T \in \mathbb{R}^{d+1}$ is the input
- $f(\cdot)$ is the identity activation $f(z) = z$

Vectorized form: for all predictions

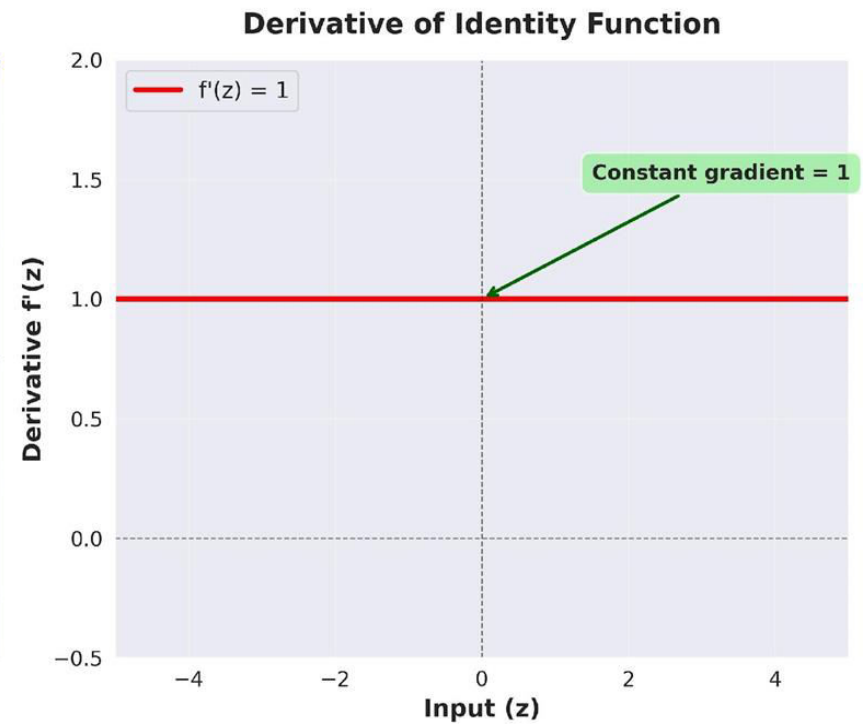
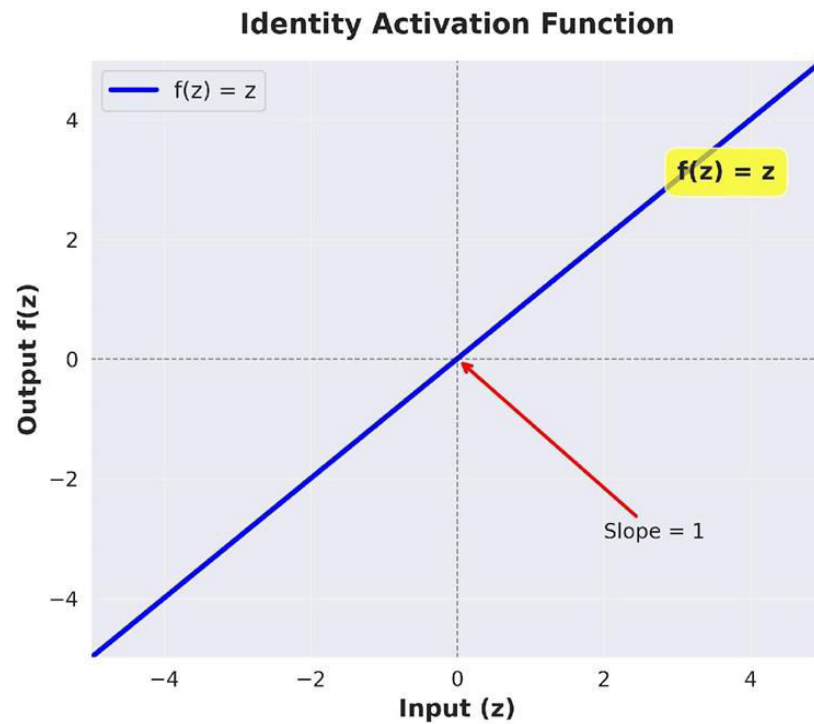
$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} \quad (3)$$

IDENTITY ACTIVATION FUNCTION

Identity Activation Function

$$f(z) = z$$

(4)



IDENTITY ACTIVATION FUNCTION

❖ Properties of Identity Function

- Output: Continuous, $(-\infty, \infty)$
- Differentiable everywhere: $f'(z) = 1$
- Allows output to take any real value
- Gradient flows unchanged through the neuron
- Perfect for predicting continuous targets

❖ Why identity for regression?

- Target values are continuous (prices, temperatures, etc.)
- Need to predict any real number, not just binary $\{0, 1\}$
- Differentiability enables gradient-based optimization
- No information loss from input to output

OBJECTIVE FUNCTION: SQUARED ERROR LOSS

We need to measure how well our model fits the data.

Loss for single example:

$$\ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) = \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{2} (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)})^2 \quad (5)$$

Total Loss (Mean Squared Error):

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \frac{1}{2} (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)})^2 \quad (6)$$

Goal: Find $\mathbf{w}^*$ that minimizes $J(\mathbf{w})$

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} J(\mathbf{w}) \quad (7)$$

WHY SQUARED ERROR?

Advantages of squared error loss:

- **Differentiable:** Smooth everywhere, easy to optimize
- **Convex:** Single global minimum (for linear models)
- **Penalizes large errors:** Quadratic growth
- **Statistical interpretation:** Maximum likelihood under Gaussian noise

Vector Form:

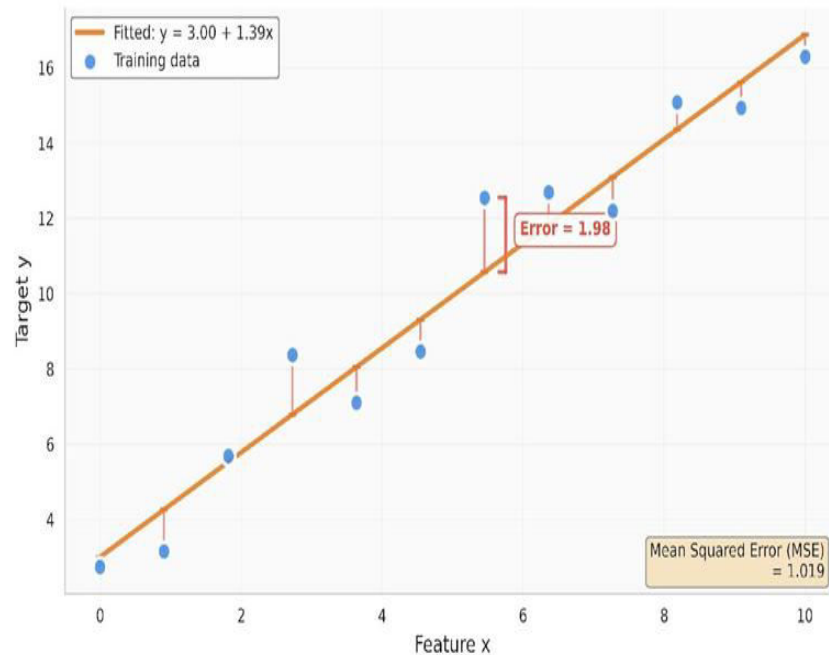
$$J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \frac{1}{2N} (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

where $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$ is the design matrix and $\mathbf{y} \in \mathbb{R}^N$ is the target vector.

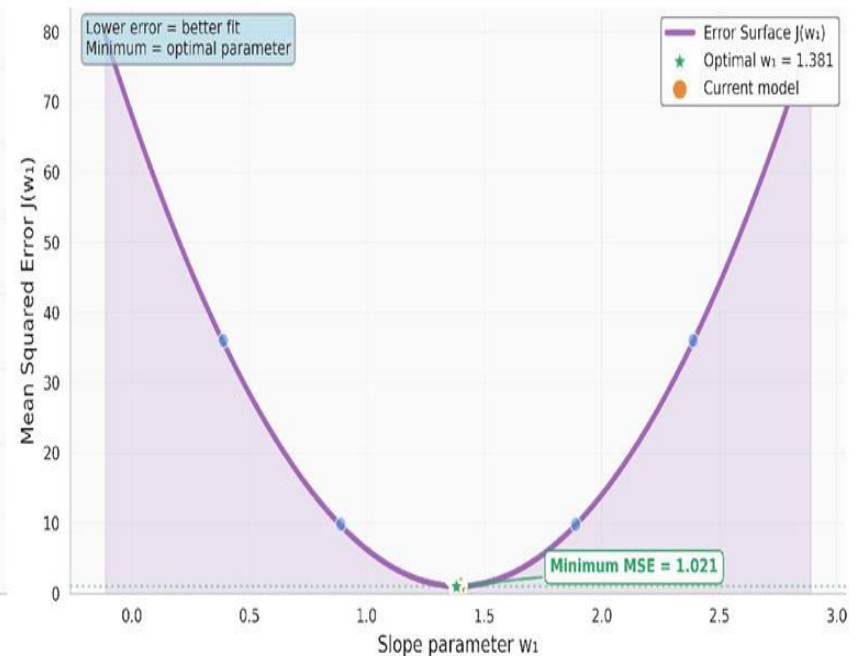
VISUALIZING THE ERROR SURFACE

Understanding Error Surface: From Individual Errors to Loss Function

(a) Linear Regression with Prediction Errors
Vertical lines show error for each data point



(b) 1D Error Surface
How error changes with slope parameter

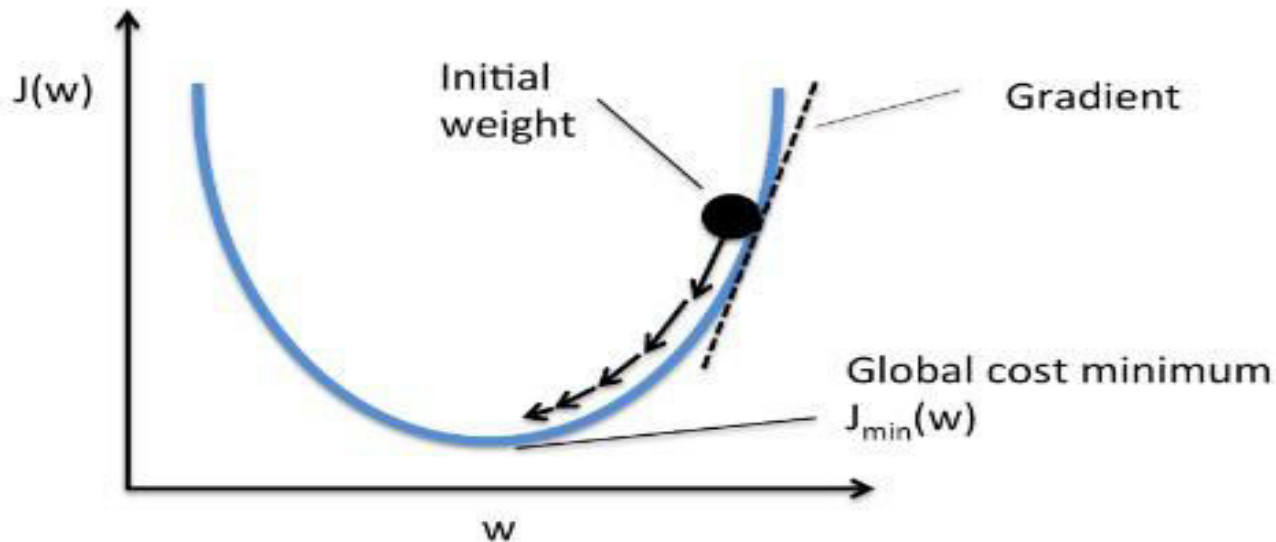


← Sum of all squared vertical errors (left) creates the parabolic error surface (right) →

- The error surface is **convex** with a single global minimum.

GRADIENT DESCENT ALGORITHM

Idea: Iteratively update weights in the direction of steepest descent



where $\eta > 0$ is the **learning rate** (step size).

Intuition: Move downhill on the error surface to find the minimum.

BATCH GRADIENT DESCENT ALGORITHM

Algorithm 1: Gradient Descent for Linear Regression

Input: Dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$, learning rate η , tolerance ϵ

Output: Learned weights $\mathbf{w}$

```
1 Initialize  $\mathbf{w}^{(0)} = \mathbf{0}$  (or random);
2  $t \leftarrow 0$ ;
3 while not converged do
4   | Compute gradient:  $\nabla J(\mathbf{w}^{(t)}) = \frac{1}{N} \mathbf{X}^T (\mathbf{X} \mathbf{w}^{(t)} - \mathbf{y})$ ;
5   | Update weights:  $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla J(\mathbf{w}^{(t)})$ ;
6   | if  $\|\nabla J(\mathbf{w}^{(t+1)})\| < \epsilon$  then
7   |   | break // Converged
8   |  $t \leftarrow t + 1$ ;
9 return  $\mathbf{w}^{(t+1)}$ ;
```

COMPUTING THE GRADIENT

Gradient of squared error loss:

$$\nabla J(\mathbf{w}) = \frac{\partial J}{\partial \mathbf{w}} = \frac{1}{N} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}^{(i)} - y^{(i)}) \mathbf{x}^{(i)} \quad (9)$$

Matrix Form:

$$\nabla J(\mathbf{w}) = \frac{1}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y}) \quad (10)$$

where $\mathbf{X}^T \in \mathbb{R}^{(d+1) \times N}$, $\mathbf{X}\mathbf{w} - \mathbf{y} \in \mathbb{R}^N$, so $\nabla J \in \mathbb{R}^{d+1}$

GRADIENT DESCENT UPDATE RULE

Update equation:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \frac{\eta}{N} \mathbf{X}^T (\mathbf{X} \mathbf{w}^{(t)} - \mathbf{y}) \quad (11)$$

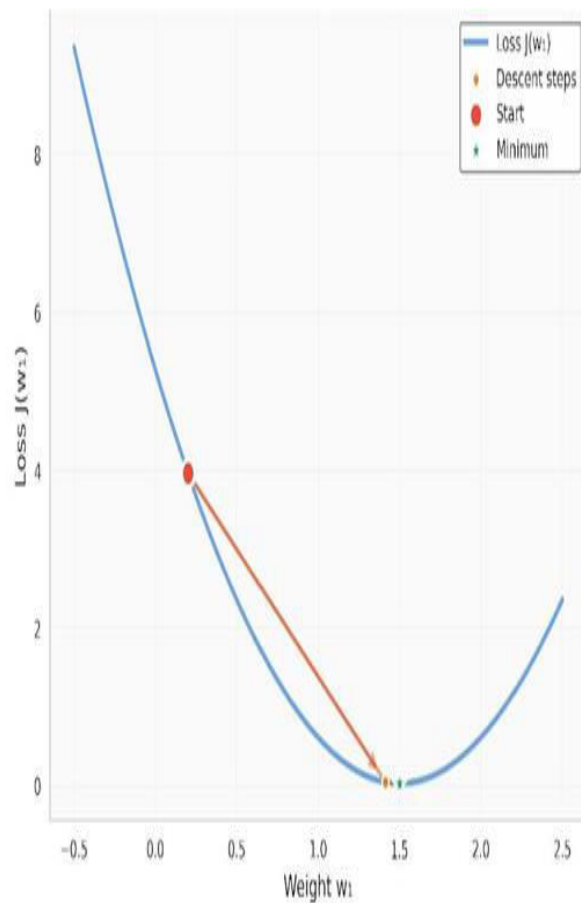
Key Parameters:

- Learning rate η : Controls step size
 - ▶ Too large: May overshoot minimum
 - ▶ Too small: Slow convergence
- Stopping criterion: $\|\nabla J(\mathbf{w})\| < \epsilon$ or max iterations

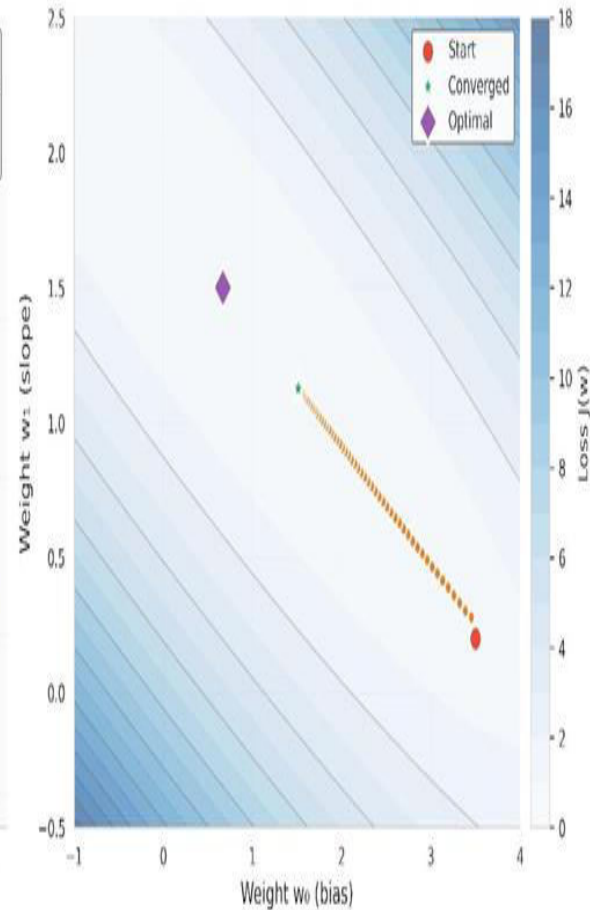
VISUALIZING GRADIENT DESCENT

Understanding Gradient Descent: Progressive Visualization from 1D to 3D

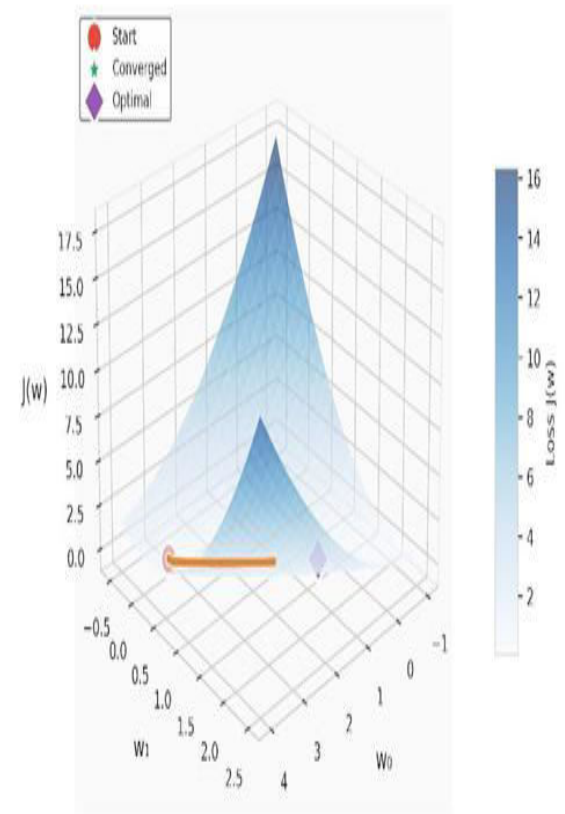
(a) 1D: Single Parameter
Intuitive gradient descent



(b) 2D: Two Parameters
Generalization to multiple dimensions



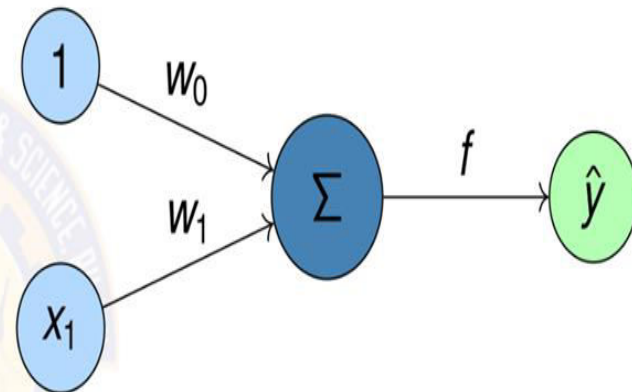
(c) 3D: Complete Surface
Full optimization landscape



NUMERICAL EXAMPLE: SETUP

Tiny Dataset: Predict house price from size

Size (x_1)	Price (y)
1	2
2	4
3	5



Matrix Formulation:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \in \mathbb{R}^{3 \times 2}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} \in \mathbb{R}^3, \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} \in \mathbb{R}^2$$

NUMERICAL EXAMPLE: INITIALIZATION

Hyperparameters: $\eta = 0.1$ $N = 3$

Initialize weights: $\mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Vectorized Prediction: $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$

Initial predictions: $\hat{\mathbf{y}}^{(0)} = \mathbf{X}\mathbf{w}^{(0)} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$

Loss Function: $J(\mathbf{w}) = \frac{1}{2N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$

Initial loss: $J(\mathbf{w}^{(0)}) = \frac{1}{6} \left\| \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} \right\|^2 = \frac{1}{6}(4 + 16 + 25) = 7.5$

NUMERICAL EXAMPLE: COMPUTING GRADIENT

Error vector: $\mathbf{e}^{(0)} = \hat{\mathbf{y}}^{(0)} - \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} -2 \\ -4 \\ -5 \end{bmatrix}$

Gradient computation: $\nabla J(\mathbf{w}^{(0)}) = \frac{1}{N} \mathbf{X}^T \mathbf{e}^{(0)}$

$$\begin{aligned} \nabla J(\mathbf{w}^{(0)}) &= \frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} -2 \\ -4 \\ -5 \end{bmatrix} \\ &= \frac{1}{3} \begin{bmatrix} -2 - 4 - 5 \\ -2 - 8 - 15 \end{bmatrix} = \begin{bmatrix} -3.67 \\ -8.33 \end{bmatrix} \end{aligned}$$

NUMERICAL EXAMPLE: WEIGHT UPDATE

Gradient descent update: $\mathbf{w}^{(1)} = \mathbf{w}^{(0)} - \eta \nabla J(\mathbf{w}^{(0)})$

Computing new weights: $\mathbf{w}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -3.67 \\ -8.33 \end{bmatrix} = \begin{bmatrix} 0.367 \\ 0.833 \end{bmatrix}$

New predictions: $\hat{\mathbf{y}}^{(1)} = \mathbf{X}\mathbf{w}^{(1)}$

$$\hat{\mathbf{y}}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 0.367 \\ 0.833 \end{bmatrix} = \begin{bmatrix} 1.20 \\ 2.03 \\ 2.87 \end{bmatrix}$$

NUMERICAL EXAMPLE: NEW LOSS

New error vector: $\mathbf{e}^{(1)} = \hat{\mathbf{y}}^{(1)} - \mathbf{y} = \begin{bmatrix} 1.20 \\ 2.03 \\ 2.87 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} -0.80 \\ -1.97 \\ -2.13 \end{bmatrix}$

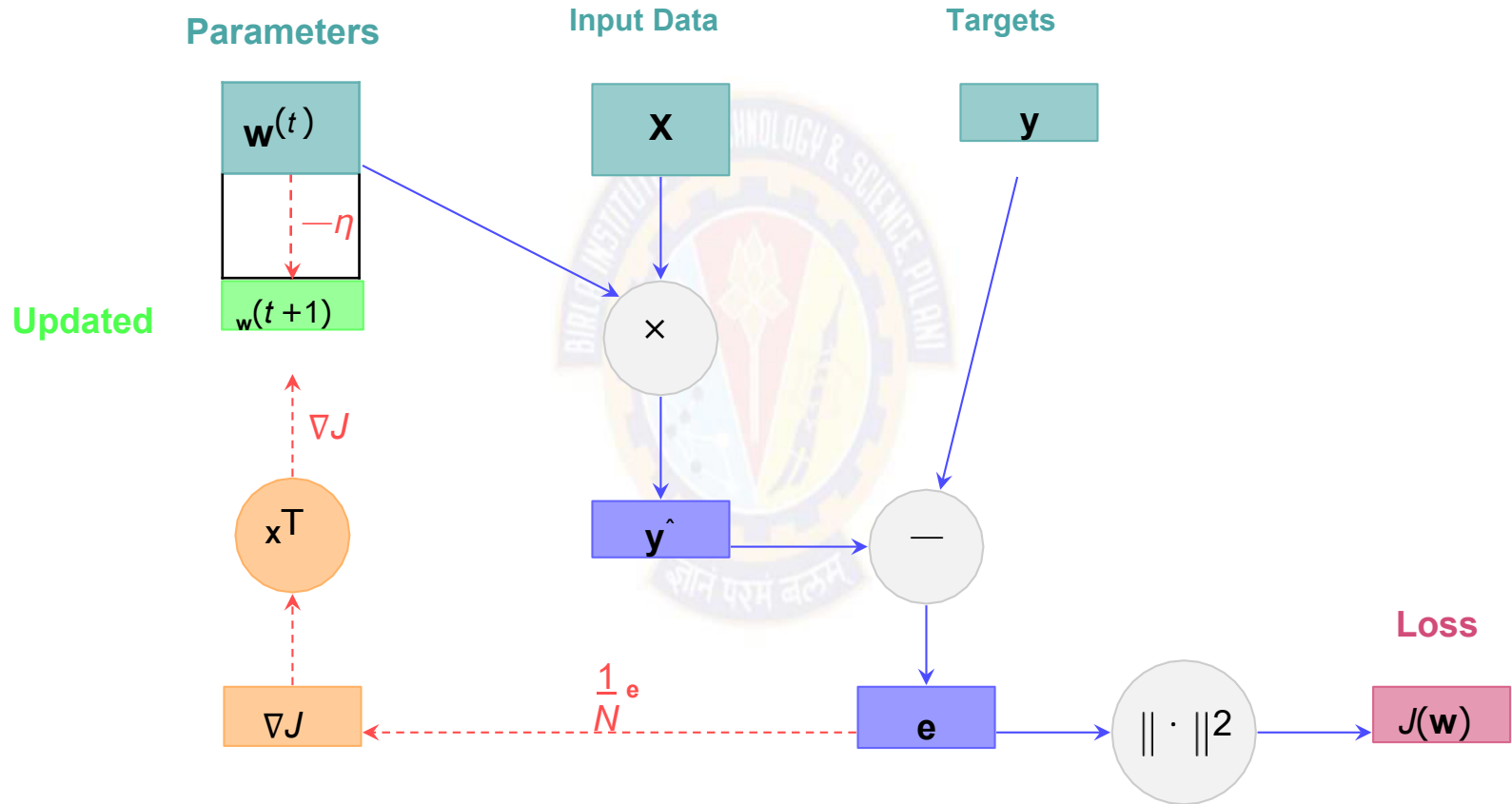
Loss after one iteration: $J(\mathbf{w}^{(1)}) = \frac{1}{6} \|\mathbf{e}^{(1)}\|^2 = \frac{1}{6} (0.64 + 3.88 + 4.54) \approx 1.51$

Progress:

- Initial loss: $J(\mathbf{w}^{(0)}) = 7.5$
- After 1 iteration: $J(\mathbf{w}^{(1)}) = 1.51$
- **80% reduction!** ✓

Continue until $\|\nabla J(\mathbf{w})\| < \epsilon$ or max iterations reached

COMPUTATIONAL GRAPH FOR ONE GRADIENT



MODEL EVALUATION: TRAINING VS TEST ERROR

Key Principle: Evaluate on unseen data

Data Split:

- Training set: Used to learn weights $\mathbf{w}$ (typically 90-99%)
- Test set: Used to evaluate generalization (typically 1-10%)

Training Error:
$$J_{\text{train}} = \frac{1}{N_{\text{train}}} \sum_{i \in \text{train}} (\hat{y}^{(i)} - y^{(i)})^2$$

Test Error:
$$J_{\text{test}} = \frac{1}{N_{\text{test}}} \sum_{i \in \text{test}} (\hat{y}^{(i)} - y^{(i)})^2$$

Goal: Minimize test error (generalization performance)

EVALUATION METRICS

1. Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2 \quad (12)$$

2. Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}^{(i)} - y^{(i)})^2} \quad (13)$$

3. Mean Absolute Error (MAE): (Less sensitive to outliers than MSE)

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y^{(i)} - \hat{y}^{(i)}| \quad (14)$$

R^2 SCORE (COEFFICIENT OF DETERMINATION)

Definition: Proportion of variance explained by the model

$$R^2 = 1 - \frac{\sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^N (y^{(i)} - \bar{y})^2} = 1 - \frac{SS_{\text{residual}}}{SS_{\text{total}}} \quad (15)$$

where $\bar{y} = \frac{1}{N} \sum_{i=1}^N y^{(i)}$ is the mean.

Interpretation:

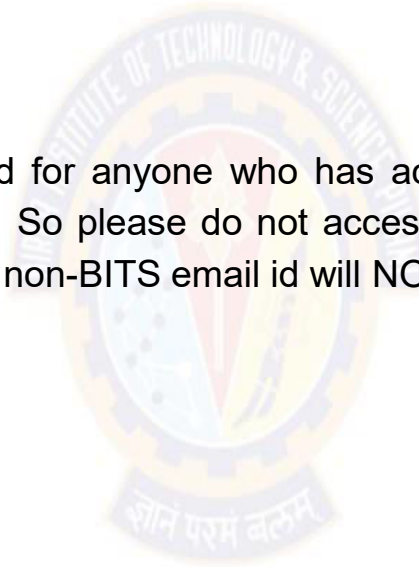
- $R^2 = 1$: Perfect fit (all variance explained)
- $R^2 = 0$: Model no better than predicting mean
- $R^2 < 0$: Model worse than baseline (possible on test set)
- Typical: $0.7 < R^2 < 0.9$ indicates good fit

PERCEPTRON DEMO PYTHON CODE

<https://colab.research.google.com/drive/10whoZYodz1H02mbVwsmTudwxaU18ukaI?usp=sharing>

Student pl note:

The Python notebook is shared for anyone who has access to the link and the access is restricted to **use BITS email id**. So please do not access from non-BITS email id and send requests for access. Access for non-BITS email id will NOT be granted.



TIPS FOR IMPLEMENTATION

1. Choosing Learning Rate η :

- Start with $\eta \in \{0.001, 0.01, 0.1, 1.0\}$
- Plot loss vs iterations for each
- If loss increases or becomes NaN $\rightarrow$ decrease η
- If convergence too slow $\rightarrow$ increase η
- Use learning rate schedules for fine-tuning

2. When to Stop Training:

- Gradient magnitude: $\|\nabla J\| < \epsilon$ (e.g., $\epsilon = 10^{-4}$)
- Loss change: $|J^{(t)} - J^{(t-1)}| < \epsilon$
- Maximum iterations reached (e.g., $T = 1000$ or $10,000$)
- Validation loss starts increasing (early stopping for overfitting)

MORE PRACTICAL TIPS

3. Weight Initialization:

- Zero initialization: $\mathbf{w}^{(0)} = \mathbf{0}$ (works well for linear regression)
- Avoid large initial values (can cause divergence or overflow)

4. Feature Engineering:

- Domain knowledge: Create meaningful derived features (e.g., price per sq ft)
- Feature selection: Remove irrelevant/redundant features

5. Feature Scaling and Normalization:

- Features with different scales can slow convergence
- 1. Min-Max Scaling: Scale to $[0, 1]$ $x'_j = \frac{x_j - \min(x_j)}{\max(x_j) - \min(x_j)}$
- 2. Standardization (Z-score): Zero mean, unit variance $x'_j = \frac{x_j - \mu_j}{\sigma_j}$

DEBUGGING CHECKLIST

Common Issues and Solutions:

- Loss is NaN/Inf:

- ▶ Learning rate too large
- ▶ Numerical overflow (check feature scales)
- ▶ Solution: Decrease η , scale features

- Training works but test error high:

- ▶ Overfitting (too many features relative to data)
- ▶ Solution: Regularization, more data, or simpler model

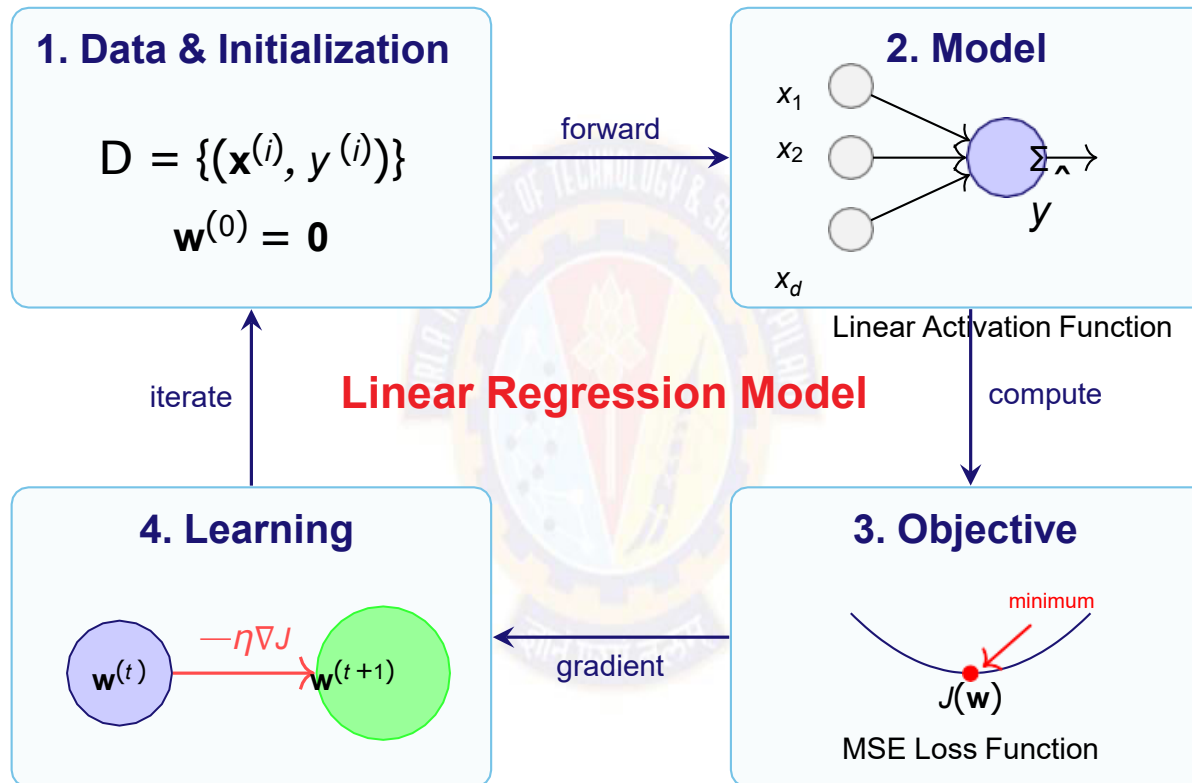
- Loss oscillating:

- ▶ Learning rate too large
- ▶ Solution: Decrease η or use learning rate decay

- Loss not decreasing:

- ▶ Learning rate too small
- ▶ Bug in gradient computation (check signs!)
- ▶ Wrong sign in update rule (should be minus)
- ▶ Solution: Increase η , verify code

SUMMARY: PUTTING IT ALL TOGETHER



This framework extends to more complex models: neural networks, deep learning!

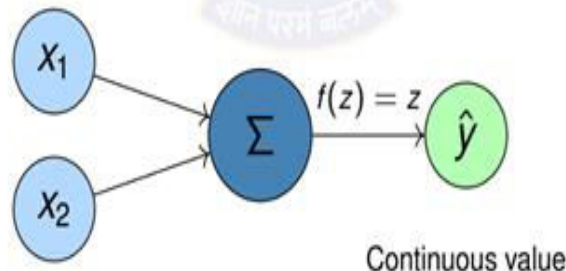
KEY TAKEAWAYS

- Regression predicts continuous outputs from input features.
- Linear regression assumes a linear relationship: $y = \mathbf{w}^T \mathbf{x} + w_0$
- A single neuron with identity activation implements linear regression.
- Squared error loss provides a convex objective function.
- Gradient descent iteratively optimizes weights: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$
- Feature scaling is critical for fast convergence.
- Evaluation on test set ensures generalization.

LINEAR REGRESSION

Key insight: Linear relationship between inputs and output, optimized by minimizing squared errors.

- **Problem:** Predict continuous values: $y \in \mathbb{R}$
- **Model:** Single neuron with identity activation
- **Output:** Predicted value $\hat{y} = \mathbf{w}^T \mathbf{x} \in (-\infty, \infty)$
- **Loss:** Mean Squared Error (MSE)
- **Training:** Batch Gradient Descent (GD)



REFERENCES

- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into Deep Learning*. Cambridge University Press. Chapter 3 (Linear Neural Networks for Regression). <https://d2l.ai/>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 5 (Machine Learning Basics). <https://www.deeplearningbook.org/>
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. Chapter 3 (Linear Models for Regression).
- Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press. Chapter 11 (Linear Regression). <https://probml.github.io/pml-book/>

Thank You!